

Python Foundations for Data and AI Work

Good AI work depends on ordinary software discipline: clear functions, validated inputs, tests, dependency management, and reproducible data transformations. Learn Python and numerical tools through small tasks whose expected result you can compute independently.

For: Developers moving toward data analysis or ML who need reliable Python, structured data handling, numerical reasoning, and reproducible experiments before building models.

Suggested pace: 4 weeks of small reproducible exercises | Beginner Python

What you will be able to do

- Write clear functions using collections, exceptions, and standard-library tools.
- Read and validate CSV or JSON data without losing type or missing-value meaning.
- Understand array shapes, vectorized operations, and numerical pitfalls.
- Create reproducible experiments with isolated environments and explicit data splits.

Build the right mental model

Python is not just notebook cells

Use functions and modules for reusable logic, with explicit inputs and outputs. Notebooks are useful for exploration, but hidden execution order can make results irreproducible. A fresh run should reconstruct the result without relying on stale variables.

Data contracts precede transformations

Specify columns, units, keys, types, and missing-value meaning. Use structured parsers rather than splitting strings by punctuation. Validate before aggregating; a silently malformed date or duplicate key can invalidate an otherwise correct model.

Shapes and precision matter

Broadcasting can perform an unintended calculation while producing a plausible array. Check dimensions and axis meaning. Floating-point arithmetic is approximate; compare with tolerances where appropriate and use stable formulations for extreme values.

Study with evidence.

Use synthetic or public data with a known small expected result. Keep environment setup, data provenance, code, and tests together. After exploration, restart the environment and run everything from the beginning to expose hidden state.

Week 1: Language and structure

Practice functions, lists, dictionaries, sets, iteration, exceptions, pathlib, and tests. Write a small command-line data validator. Separate parsing errors from domain errors and return useful messages.

Deliverable and check: A script that handles valid, empty, missing, and malformed inputs predictably.

Week 2: Structured data

Read CSV and JSON with standard parsers, normalize types deliberately, and handle duplicates and missing values. Learn basic dataframe grouping and joins while preserving row grain and identifiers.

Deliverable and check: A small data-quality report with explicit rules and examples of rejected or repaired rows.

Week 3: Numerical arrays

Practice vector shapes, matrix multiplication, reductions, broadcasting, and random seeds. Compare a vectorized calculation with a simple loop on small data. Check units and avoid accidental integer truncation.

Deliverable and check: Tests that catch shape mismatch and confirm numerical results within a justified tolerance.

Week 4: Reproducible analysis

Create an isolated environment, record dependency versions, split data before fitted preprocessing, and save experiment configuration. Move core logic out of exploratory cells and run the analysis from a clean start.

Deliverable and check: A reproducible mini-analysis with a baseline, test fixtures, and clearly separated training and evaluation data.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Build a leak-free standardization step

You have a numeric feature and want to standardize it before modeling. The dataset is split into training and evaluation rows. Using all rows to compute the mean would allow evaluation information into preprocessing.

A worked approach

1. Compute the mean and scale from training rows only, after defining how missing values are handled. Store those fitted values as transformation state; do not recompute them independently for the evaluation set.
2. Apply the same transformation to training and evaluation rows. Test a constant training feature and choose a documented behavior for zero scale rather than dividing by zero or silently producing invalid values.
3. Compare the output with a small hand-calculated example and with a maintained preprocessing library. Wrap the step in a pipeline for cross-validation so each fold fits only on its own training portion.

Expected result

The preprocessing contract is reproducible and respects the evaluation boundary. A correct mathematical formula is not enough if its fitted state used data that would be unavailable during deployment.

Verify the result

- Are fitted statistics derived only from the intended training subset?
- Are zero scale and missing values handled explicitly?
- Can a fresh process reproduce the transformed data?

Extend the exercise

Add a categorical feature and an unseen evaluation category. Define whether it maps to an unknown bucket, is ignored, or is rejected, and test the choice before training a model.

Make the learning stick.

Is vectorized code always faster?

Not necessarily. It can reduce interpreter overhead, but may create large temporary arrays or use an unsuitable algorithm. Measure representative data and memory use, and retain a simple reference implementation for correctness.

Why does notebook execution order matter?

A cell may depend on a variable created or modified by an earlier manual action that is not reflected in top-to-bottom order. Restarting and rerunning exposes these hidden dependencies and improves reproducibility.

Is a random seed enough for reproducibility?

No. Data versions, code, dependencies, hardware behavior, parallelism, and external services can also matter. Record the relevant configuration and distinguish exact reproducibility from statistically comparable results.

Avoid these traps

- Parsing structured data with fragile string splitting.
- Accepting unintended broadcasting because no exception occurred.
- Fitting preprocessing on the complete dataset before evaluation.

Ready for the next step?

- Functions and data contracts are explicit and tested.
- Array shapes and units can be explained.
- A clean run reproduces the analysis.
- Training and evaluation boundaries remain separate.

Reliable next reads

- [Python tutorial](#)
- [NumPy user guide](#)
- [scikit-learn common pitfalls](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.